

# Intelligent dead reckoning internals

How every layer works, what each measurably contributes, and whether a partial GNSS fix of fewer than four satellites can still be used.

---

Two parts. Part one walks the pipeline layer by layer, giving for each the inputs it takes, the transform it applies, and what it was measured to do. Part two answers the satellite question, which bears directly on the weakest channel in the system. Every figure is measured on recorded data through the harness in eval/, not estimated.

## Part I — the processing layers

---

WORKING

### 1. Sensor acquisition

in: phone hardware -> out: eleven timestamped streams

Eleven streams at explicit rates: accelerometer and gyroscope at 200 Hz with their uncalibrated variants, magnetometer and rotation vector at 50 Hz, game rotation vector and linear acceleration at 100 Hz, gravity at 50 Hz, barometer at 25 Hz. The uncalibrated variants are recorded deliberately: a filter that estimates its own bias states wants the raw signal, not one the operating system has already corrected.

The sensor hub batches up to one second into its hardware FIFO before waking the application processor. Batched events keep their own correct timestamps, so batching costs latency but never accuracy.

Everything is stamped on elapsedRealtimeNanos, the same monotonic clock SensorEvent.timestamp uses. That clock does not jump when NTP corrects the wall clock, which is what makes a reliable dt possible.

---

WORKING

### 2. Threading contract

in: all callbacks -> out: one serialised write path

One thread, imu-logger, owns every sensor callback, every location and GNSS callback, every file write, and the integrator. Because there is exactly one writer, no lock appears anywhere in the recording path.

Two other threads exist only because their work would stall a 200 Hz path. imu-ml holds the model, whose forward pass costs about 40 ms. map-match holds the road reader, whose first read of a new area parses several hundred ways off disk. Both receive posted work and post their results BACK to the logger thread rather than writing anything themselves.

---

LARGEST ERROR SOURCE

### 3. DeadReckoner — strapdown inertial integration

in: accel 200 Hz, gyro, rotation vector -> out: position, velocity, drift

The rotation vector gives a device-to-world matrix in ENU convention. Each accelerometer sample is rotated into world axes, gravity is removed from the vertical, a learned bias is subtracted, and the result is integrated twice. A sample whose dt exceeds 50 ms is dropped rather than integrated, because a gap means the FIFO stalled and integrating across it would fabricate a velocity step.

Stand-still is detected from acceleration norm and angular rate, and during it three things happen: gravity magnitude is learned, world-frame bias is learned, and velocity is forced to zero. Learning the gravity magnitude

is what absorbs the accelerometer's scale-factor error, which on this handset is about 0.9%.

**Measured defect.** Free-running without GNSS, this layer recovers about 30% less distance than was actually travelled: 15.0% short at 10 s, 21.3% at 30 s, 29.5% at 60 s. The zero-velocity update was the leading suspect and is NOT the cause - it fires on 0 to 2% of samples during outages, and at 30 s the shortfall is 21.3% while it fires on 0.0% of them. Dropped samples are 0.0%. The actual cause is a systematic deceleration: over 46 sixty-second outages, acceleration projected onto the direction of travel averages  $-0.0195 \text{ m/s}^2$  and is negative in 89% of them, which over 60 s removes 1.17 m/s. The structural fault is that horizontal bias states are learned only during stand-still, so they are estimated exactly when they are not needed and frozen exactly when they are.

#### WEAK CHANNEL

## 4. ResNet1D — the speed model

in: (B, 6, 100), 10 s at 10 Hz -> out: four scalars

Six channels: vehicle-frame accelerometer forward, lateral, vertical, then gyroscope on the same axes. A strided stem convolution halves time, then four residual stages each double channels and halve time again, taking 6x100 to 512x7. Global average pooling then four linear heads.

There is deliberately no max-pool after the stem. At 10 Hz a 100-sample window is already short, and pooling twice early would discard the temporal detail the estimate depends on.

| STAGE       | OUTPUT       | PARAMETERS | SHARE |
|-------------|--------------|------------|-------|
| stem, k7 s2 | (B, 64, 50)  | 2,816      | 0.1%  |
| stage 0     | (B, 64, 50)  | 49,664     | 1.3%  |
| stage 1     | (B, 128, 25) | 181,504    | 4.7%  |
| stage 2     | (B, 256, 13) | 723,456    | 18.8% |
| stage 3     | (B, 512, 7)  | 2,888,704  | 75.1% |
| four heads  | (B, 1) each  | 2,052      | 0.1%  |

The heads are mu (speed, through softplus so it cannot go negative), logvar (clamped to [-6, 4], which weights the fusion gain downstream), stationary\_logit, and yaw\_rate.

**Measured limits.** Test RMSE 4.88 m/s against 6.84 for predicting a constant, correlation +0.74. Held out by DRIVER rather than journey that falls to 5.66 against 6.89, a 17.8% margin rather than the 28.7% a journey split suggests, because held-out journeys share roads with training ones by up to 50% bounding-box overlap.

Train RMSE is 0.396 with correlation 0.999, a twelvefold train-to-test gap - but shrinking the network does not close it, so this is distribution shift between sessions, not classic variance. Everything tried (more data, less capacity, dropout, weight decay, two physics losses, three augmentations, quality weighting) moves test error by at most a couple of percent.

**The yaw head should not be used for navigation.** It correlates 0.82 to 0.83 with true yaw rate where the raw gyro axis it was trained from correlates 0.94 to 0.996, and carries enough bias to swing heading 893 to 1410 deg/hr.

#### WORKING

## 5. MapMatcher — online HMM

in: drifting position, course, drift estimate -> out: snapped position, road bearing

States are candidate positions on nearby road segments; observations are the dead-reckoned fixes. Viterbi runs incrementally with a beam of 12 over at most 24 candidates, so each fix costs beam x candidates rather than a re-run over the history.

This departs from Newson and Krumm deliberately. Their transition term is the difference between straight-line distance and route distance through the road graph, which needs connectivity Mapsforge does not provide. What replaces it is a heading term, and that is the point rather than a consolation: the whole reason this layer exists is that heading integration is where dead reckoning fails, so scoring a candidate on how well the road's bearing agrees with the vehicle's course is precisely the observation the IMU cannot supply for itself.

Emission spread is 30 m, far wider than the 4.07 m Newson and Krumm fit for GPS, because the observation here is a dead-reckoned position that may be hundreds of metres out, and too tight a sigma would reject the correct road outright.

The correction budget is the safety valve: a match is rejected if it would move the estimate further than  $\max(\text{drift}, 12 \text{ m})$ . Measured across four sessions, snapping at about 37 m drift cut position error by 30%, but with drift near zero it made things 40 times worse by dragging a fix 0.6 m from truth onto a road 25 m away. Being unaided is not the same as being wrong yet.

BUILT, GATED OFF

## 6. RoadGraph and AlongRoadTracker

in: Mapsforge geometry -> out: connected topology, along-road position

Mapsforge stores ways clipped per tile with no node identity, which is why the conventional answer is to route against a server. But coordinates are stored in microdegrees, about 0.11 m, so segments that once shared an OpenStreetMap node come back on identical coordinates. Snapping endpoints onto a half-metre grid rebuilds the graph entirely on the device.

| NETWORK               | SEGMENTS | NODES   | LINKED | LARGEST COMPONENT |
|-----------------------|----------|---------|--------|-------------------|
| IIT Kharagpur         | 25,383   | 18,570  | 100.0% | 98.5%             |
| Coventry - Birmingham | 155,050  | 148,363 | 99.99% | 99.0%             |

With topology recovered, position can advance as a scalar walked along a polyline: the road supplies direction and heading error stops entering the answer at all. Fed a trustworthy distance this cuts 60 s outage error by 42%. Fed the distance the integrator currently produces it is 20% WORSE than doing nothing, which is why it ships disabled.

WORKING

## 7. Offline pipeline

in: IO-VNBD S and V file pairs -> out: training set, evaluation

eval/iovnbd.py pairs each smartphone stream with its vehicle reference stream. Six things in those files contradict their own headers, each capable of silently poisoning everything downstream: a speed column labelled km/h that holds m/s, a gravity column that is a constant placeholder, an orientation column that contradicts the accelerometer, GPS held for 9 s at a stretch, synchronised files that are not time-aligned, and gyroscope columns permuted relative to the accelerometer.

Alignment is two-stage and the second stage is not optional. A coarse lag from the 9-second-held phone GPS can only localise to its own step; the residual measured 4.4 s, at which no linear combination of phone acceleration predicted vehicle acceleration ( $r = 0.06$ ). Refining against two genuine 10 Hz signals took it to  $r = 0.45$ .

eval/ahrs.py supplies attitude, since the dataset has none usable. It uses only what can be checked: tilt from the accelerometer, yaw from the gyro projected onto the accelerometer-derived vertical. The vehicle's own yaw rate aligns and scores it but never builds it, so it stays a held-out validation signal rather than becoming part of the input it is meant to supervise.

## Part II — using fewer than four satellites

Short answer: **yes, and it targets exactly the channel that is currently broken** - but it belongs in the estimator, not in the neural network.

### Why four, and what fewer still contains

A standalone position fix has four unknowns: three coordinates plus the receiver clock bias, unknown because a phone's oscillator is nowhere near good enough to keep GPS time. Each pseudorange gives one equation, so four satellites is the minimum for a snapshot fix. That is a constraint on solving for POSITION, and it says nothing about whether one, two or three satellites carry information. They plainly do - each is still a real measurement of a real geometric quantity.

- **Pseudorange** constrains position and clock bias jointly. It needs three more friends to be useful on its own.
- **Pseudorange rate**, the Doppler, constrains VELOCITY and clock drift. This is the interesting one.

The current code discards both. SensorService records only GnsStatus, and treats satellitesUsedInFix below 4 as unusable, at which point the integrator free-runs as though the sky were empty.

### Why Doppler is the right target here

Every measurement in part one points the same way: after heading is taken from the gyro rather than the model, **speed is the binding constraint**. The model manages 4.88 m/s RMSE, the integrator runs 30% short on distance, and free-running drift is 46% of distance travelled at 300 s.

A pseudorange rate is a direct measurement of velocity projected onto the line of sight, and Android reports its own uncertainty through getPseudorangeRateUncertaintyMetersPerSecond, typically well under 1 m/s on modern chipsets. That is roughly an order of magnitude better than anything the speed model produces, and it requires no position fix whatsoever.

### The vehicle constraint collapses the requirement

For a general receiver, velocity has four unknowns - three components plus clock drift - so four satellites again. A road vehicle is not general. It travels along its heading, and its vertical velocity is nil:

```
v = s . [sin h, cos h, 0] h from the gyro  
rho_dot = e . (v_sat - v) + c . (drift_rx - drift_sat) + noise
```

With heading supplied by the integrator, the unknowns collapse to two: forward speed  $s$  and receiver clock drift. So **two satellites determine vehicle speed**. And because a phone's clock drift is stable over tens of seconds, coasting it from the last healthy period leaves one unknown, so **a single satellite is enough to pin speed**.

**The counterintuitive part:** which satellite matters more than how many. The measurement carries information about speed in proportion to the along-track component of the line of sight. A satellite directly overhead contributes almost nothing about horizontal speed; a low-elevation satellite ahead or behind contributes the most. That inverts the usual preference, where high elevation is favoured for less multipath.

## Implementation

- 1 **Record raw measurements.** Register a `GnssMeasurementsCallback` and write a new `gnss_raw.csv`: per satellite the `svid`, constellation, pseudorange rate and its uncertainty, accumulated delta range and its state, `C/N0`; plus `GnssClock` drift and bias. None of this is captured today, so this is step zero regardless of what is done with it.
- 2 **Cache ephemeris while healthy.** Satellite position and velocity are needed to form the line of sight and to remove the satellite's own motion, which projects to hundreds of m/s and cannot be ignored. Register `GnssNavigationMessage` and keep the broadcast ephemeris; it stays valid for hours, and a tunnel transit is minutes.
- 3 **Shortcut worth knowing.** `GnssStatus` already exposes per-satellite azimuth and elevation while tracking, which gives the line-of-sight direction without decoding ephemeris at all. Only the satellite-velocity correction still needs it.
- 4 **Solve for speed.** With one or two satellites, least-squares the equation above for  $s$ , weighting by the reported uncertainty and rejecting anything under about 20 dB-Hz as multipath-prone. Reject epochs where the along-track geometry term is near zero, since those measure nothing useful.
- 5 **Feed it as a velocity observation**, not a position one. `DeadReckoner` already has the right shape for this in `applyModelSpeed`, which blends a speed estimate into velocity magnitude and leaves direction alone. A Doppler speed enters the same way with a far smaller variance.
- 6 **Validate the sign convention empirically.** Android's pseudorange rate sign has bitten people. This project has already been caught twice by exactly this class of bug - two heading sign errors that produced valid rotation matrices and plausible tracks, just rotated. Confirm against a stretch of healthy GNSS before trusting it.

## What it is worth, honestly

In a deep tunnel there are zero satellites and this contributes nothing. Its value is at the margins, and the margins are common: tunnel entry and exit, urban canyons, underpasses, dense tree cover, multi-storey car parks - everywhere the count drops below four and the current code throws the sky away.

The mechanism that matters is bounding drift rather than fixing position. Speed error integrates into displacement, so a sporadic single-satellite Doppler that resets the speed estimate every few tens of seconds prevents the error that dominates a long outage from ever accumulating. Against a measured 46% drift at 300 s, that is a large lever.

**It does not belong in the neural network.** Pseudoranges relate to position through exact, known geometry. A convolutional network fed raw ranges would have to rediscover from data what can simply be written down, and would need far more data than this project has to do it badly. The model's job stays what it is - inertial signal to speed - and its logvar head is the right place for it to participate, by telling the filter how much to trust it against a Doppler measurement that arrives with its own stated uncertainty.

## Where it sits against everything else

| CHANGE                                                                               | MEASURED EFFECT                    | NEEDS                   |
|--------------------------------------------------------------------------------------|------------------------------------|-------------------------|
| Gyro heading instead of the model's yaw head, gyro bias removed from pre-outage data | <b>300 s drift 46% to 24%</b>      | nothing                 |
| Per-session speed offset calibration                                                 | <b>30 s drift 20.5% to 16.0%</b>   | nothing                 |
| Sub-four-satellite Doppler speed aiding                                              | <b>not yet measured</b>            | raw GNSS logging        |
| Along-road tracking                                                                  | <b>60 s error -42%</b>             | a good distance channel |
| Any training-side model change                                                       | <b>at most a couple of percent</b> | -                       |

The pattern is consistent: this system's accuracy lives in the estimator and the constraints available to it, not in the network. Sub-four-satellite aiding fits that pattern exactly, which is a good reason to expect it to work, and a good reason to measure it before believing it.

All figures measured through eval/ on recorded sessions and on IO-VNBD. Reproduce with eval/dr\_diagnostics.py, eval/model\_dr\_eval.py and eval/rank\_models.py; the full experiment log is in ml\_model/PROGRESS.md. Sub-four-satellite aiding is a proposal, not a result - it is the only item here without a measured number. 4 September 2026.